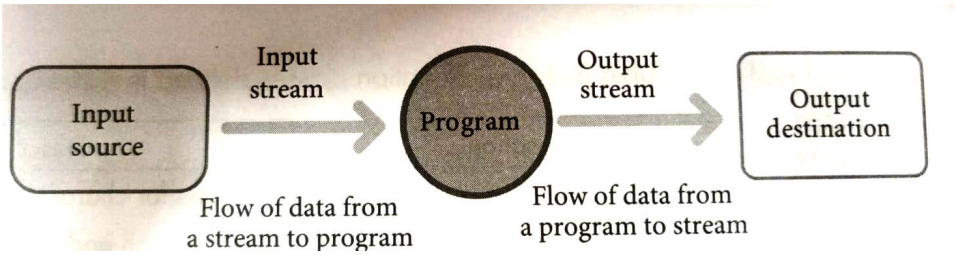
Streams:

Java treats flow of data as stream. Input and output in JAVA is based on the concept of stream. A stream is an abstract representation of an input device(produces data) or output device(consumes data).

The flow of data from a source to a Java program is called as input stream and the flow of data from a Java program to destination is called output stream. The java.io package contains a large number of stream classes to support the streams.



Standard IO Streams:

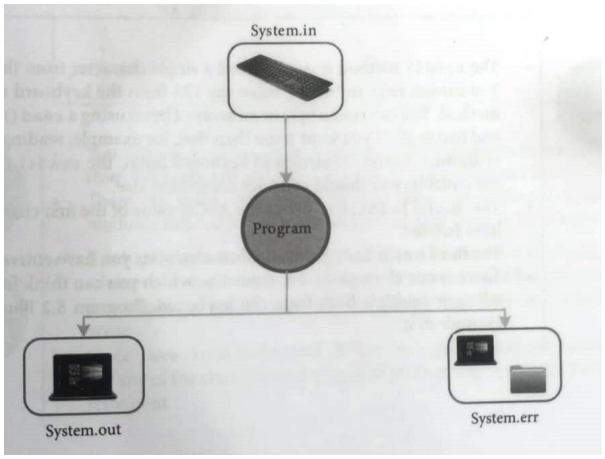
Java defines three standard streams that are accessible though members of System class.

1. **Standard input stream**: This is referred by System.in and usually corresponds to the keyboard by default. This is encapsulated by in member of System class and is of type InputStream.

2. **Standard output stream**: This is referred by System.out and usually corresponds to the console(display screen of the computer). This is encapsulated by out member of System class and is of type PrintStream.

3. **Standard error stream**: This is referred by System.err, is used to display error messages and usually corresponds to the console(display screen of the computer). This is encapsulated by err member of System class and is of type PrintStream.

SURESH YADLAPATI



Ex: Reading some data using System.in stream:

```
import java.io.*;
class StandardInputDemo {
   public static void main(String args[]) throws IOException {
      System.out.println("Enter whatever you want...");
      int i = System.in.read();
      System.out.println(i);
      System.out.println((char) i);
   }
}
```

Ex2: Reading lines of data from key board

```
import java.io.Console;

public class ConsoleDemo {
   public static void main(String[] args) {
      Console console = null;
      String name = null;

      try {
         consol = System.console();

         /*The System.console() method returns an instance  of the Console class,
           which provides methods to read user input and print text directly to the
            console.       */

           if (consol != null) {
                System.out.println(“Your name please“);
                 name = consol.readLine("Name: ");
                  System.out.println("Hi  " + name + “!”);

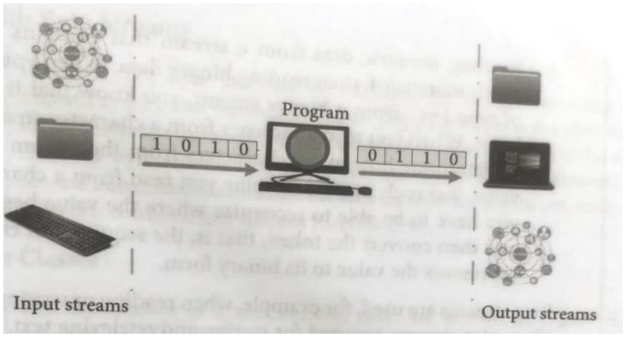
         }
      } catch (Exception ex) {
         ex.printStackTrace();
      }
   }
}
```

SURESH YADLAPATI

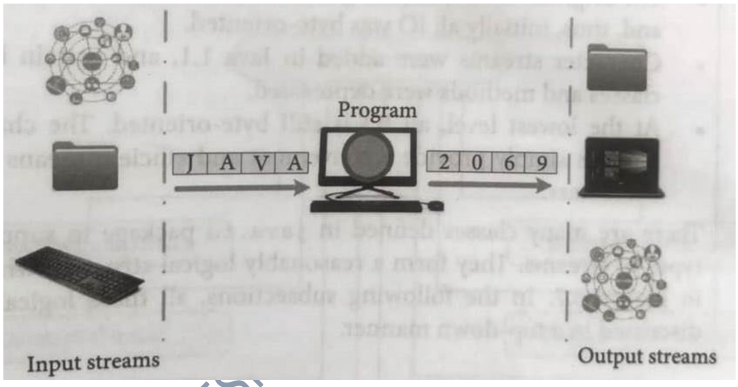
Types of Streams:

A stream can be used broadly in two ways: Byte streams and Character streams.

- Byte streams considers data in binary representation.



- Character streams read data in the form of characters as a unit. Java stores internally characters as Unicode characters which are 16 bit in size.

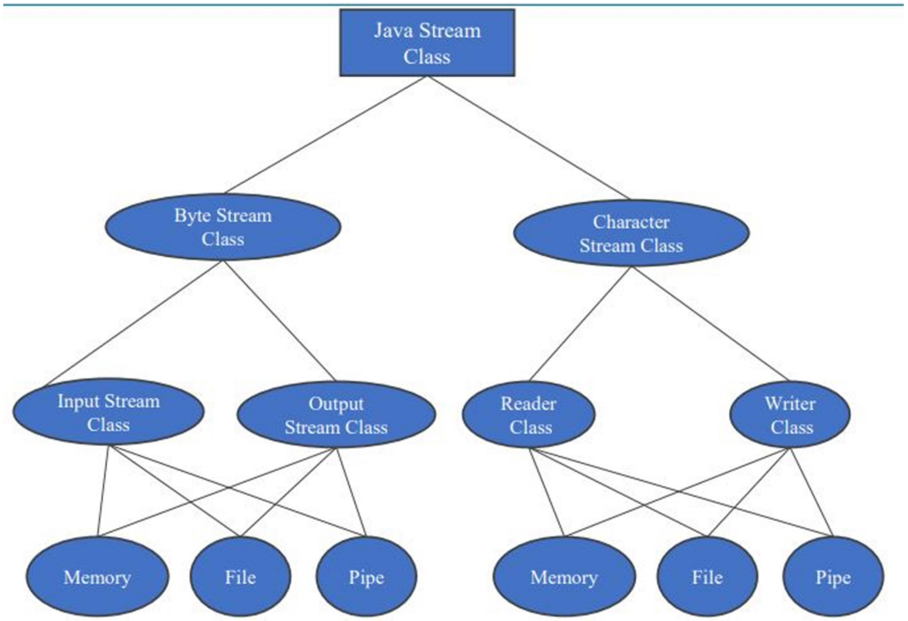


Differences between byte and character stream classes:

Aspect	**Character Streams**	**Byte Streams**
Data Handling	Handle character-based data	Handle raw binary data
Representation	Classes end with "Reader" or "Writer"	Classes end with "InputStream" or "OutputStream"
Suitable for	Textual data, strings, human-readable info	Non-textual data, binary files, multimedia
Character Encoding	Automatic encoding and decoding	No encoding or decoding
Text vs non-Text data	Text-based data, strings	Binary data, images, audio, video
Performance	Additional conversion may impact performance	Efficient for handling large binary data
Handle Large Text Files	May impact performance due to encoding	Efficient, no encoding overhead
File Handling	Read/write text files	Read/write binary files

Network Communication	Sending/receiving text data	Sending/receiving binary data
Handling Images/Audio/Video	Not designed for handling binary data directly	Suitable for handling binary multimedia data

The logical structure of java.io package is shown in the following figure:

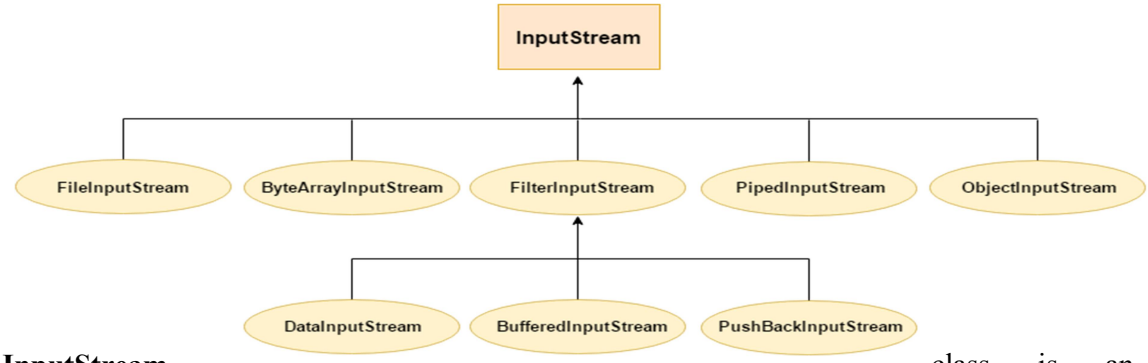


Byte Streams:

In java.io, byte streams are defined by using two class hierarchies: InputStream and OutputStream.

InputStream is the abstract base class for byte stream input operations whereas OutputStream is the base class for byte stream operations.

InputStream Classes:



InputStream class is an abstract base class for byte stream input operations. InputStream class contains the following methods which are implemented in sub classes.

- int read() - reads one byte of data from the input stream
- int read(byte[] array) - reads bytes from the stream and stores in the specified array
- int read(byte[] array, int offset, int numBytes) – Attempt to read numBytes into array starting at offset.
- int available() - returns the number of bytes available in the input stream
- void mark(int numberofbytes) - marks the position in the input stream up to which data has been read
- void reset() - returns the control to the point in the stream where the mark was set
- Boolean markSupported() - checks if the mark() and reset() method is supported in the stream
- long skips(long numberofbytes) - skips and discards the specified number of bytes from the input stream
- void close() - closes the input stream

ByteArrayInputStream: Input stream that reads from a byte array. An instance of this contains internal buffer.

FileInputStream: Input stream that reads from a file.

ObjectInputStream: For reading objects from a stream.

PipedInputStream: For reading from a piped stream like a network socket or port.

FilterInputStream: For filtering input from an existing stream.

DataInputStream: An input stream that contains methods for reading Java primitive data types from stream.

Methods in DataInputStream class:

- **boolean readBoolean()**: Reads one byte and returns true if it's nonzero, false otherwise.
- **byte readByte()**: Reads and returns one byte (8-bit signed integer).
- **char readChar()**: Reads two bytes and returns a char value (16-bit Unicode character).
- **double readDouble()**: Reads eight bytes and returns a double value.
- **float readFloat()**: Reads four bytes and returns a float value.
- **int readInt()**: Reads four bytes and returns an int (32-bit signed integer).
- **long readLong()**: Reads eight bytes and returns a long (64-bit signed integer).
- **short readShort()**: Reads two bytes and returns a short (16-bit signed integer).
- **String readUTF()**: Reads a UTF-8 encoded string.

BufferedInputStream: Reading a chunk of bytes from an internal buffer associated with a stream.

SURESH YADLAPATI

PushbackInputStream: It can unread a byte which is already read and push back one byte.

Example program 1:

```
import java.io.*;
public class InputStreamMethods {
        public static void main(String[] args) throws IOException {
      FileInputStream in = new FileInputStream("input.txt");
      // 1. read()
      System.out.println("1. read(): " + (char) in.read());
      // 2. read(byte[] b)
      byte[] b = new byte[5];
      System.out.println("2. read(byte[]):" +  in.read(b)+ ":"+new String(b));
      byte[] c = new byte[5];
      System.out.println("2. read(byte[], 0, n):" + in.read(c,0,2)+":"+new String(c));
      // 3. available()
      System.out.println("3. available(): " + in.available());
       // 4. skip(n)
      in.skip(1);
      System.out.println("4. skip(1): One byte skipped");
      System.out.println("1. read(): " + (char) in.read());
     // 5. read(byte[] b, int off, int len)
      byte[] d = new byte[10];
      System.out.println("5. read(d, 0, 2): "+ in.read(d, 0, 2)+":"+new String(d));
      // 6. markSupported()
      System.out.println("6. markSupported(): " + in.markSupported());
      // 7. close()
      in.close();
      System.out.println("7. close(): Stream closed");
   }
}
```

SURESH YADLAPATI

Example program 2:

```
import java.io.*;
public class ExampleonMark {
   public static void main(String[] args) throws Exception {
        FileInputStream file = new FileInputStream("input.txt");
      BufferedInputStream in = new BufferedInputStream(file);

      // Check mark support
      System.out.println("Mark Supported: " + in.markSupported());
      // Step 1: Read first 5 characters
```

```
        System.out.print("First Read: ");
        for (int i = 0; i < 5; i++) {
            System.out.print((char) in.read());
        }

        System.out.println();
        // Step 2: Mark current position
        in.mark(10);
        System.out.println("Mark placed!");

        // Step 3: Read next 4 characters
        System.out.print("Second Read: ");
        for (int i = 0; i < 4; i++) {
            System.out.print((char) in.read());
        }

        System.out.println();
        // Step 4: Return to marked position
        in.reset();
        System.out.println("Reset performed!");

        // Step 5: Read the same 4 characters again
        System.out.print("Third Read: ");
        for (int i = 0; i < 4; i++) {
            System.out.print((char) in.read());
        }

        System.out.println();
        in.close();
    }
}
```

input.txt

Suresh Yadlapati PVPSIT

```
F:\Courses\JAVA26-27\U
Mark Supported: true
First Read: Sures
Mark placed!
Second Read: h Ya
Reset performed!
Third Read: h Ya
```

SURESH YADLAPATI

Example Program 3:

```
import java.io.*;
class DIS_Example
{
```

```
public static void main(String args[])  throws IOException
{
 DataInputStream dis = new DataInputStream(System.in);
 System.out.println("Enter your name: ");
 String str1 = dis.readLine();
 System.out.println("I know your name is " + str1);

 System.out.println("Enter a whole number: ");

 int x = dis.readInt() ;
 System.out.println((int)x);
       /*The problem is that readInt does not behave as you might expect.
        it reads the input as  bytes i.e, 4 bytes so to avoid this use
        readLine and parse the data
       */
 System.out.println("Enter a whole number: ");

 String str2 = dis.readLine();
 System.out.println(Integer.parseInt(str2));
 dis.close();
 }
}
```

SURESH YADLAPATI

```
Enter your name:
Suresh
I know your name is Suresh
Enter a whole number:
22
842140938
Enter a whole number:
33
33
```

OutputStream Classes:

